RQuery-Zustand Deep Dive

> Comprehensive study on **what React Query and Zustand are**, **why** they matter, and **exactly how** this project wires them together—from package install to rendering behavior.

Table of contents

---

1. Problem statement

Modern React apps juggle two very different classes of state:

State type	Examples	Trait	Best fit
Server state	Leads list, paginated results, mutations	Remote, async, cacheable	**React Query**
UI state	Modal open flag, selected lead, filters	Local, synchronous, ephemeral	**Zustand**

Our lead manager needs:

- Infinite scroll without double-fetching.
- CRUD mutations with optimistic UX.
- Modal toggles that re-render only the pieces of UI that actually need it.

---

2. What is React Query?

React Query (TanStack Query) is a data synchronization library for React. It:

1. **Fetches** data via `useQuery`.
2. **Caches** responses under a `queryKey`.
3. **Reuses** cached data on re-mount, refetching in the background when stale.
4. **Mutates** server state via `useMutation`, with built-in loading/error handling.

Why use it instead of `useEffect + fetch`?

- No manual loading state wiring.
- Automatic retries on network errors.
- Cache layers that persist through navigation.
- DevTools for debugging.

---

3. Installing & bootstrapping React Query

```
cd client
npm install @tanstack/react-query axios react-hot-toast
```

Wrap your app with `QueryClientProvider` (see `client/src/main.jsx`):

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 2, // 2 minutes
      cacheTime: 1000 * 60 * 60, // 60 minutes
      refetchOnWindowFocus: false,
    },
  },
});

ReactDOM.createRoot(document.getElementById('root')).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>,
)
```

> This provider makes the query cache available to every descendant, so hooks like `useQuery` can share data and avoid duplicate requests.

---

4. How `useLeads` works (line-by-line)

File: `client/src/hooks/useLeads.js`

```
export function useLeads(page = 0, size = 3) {
  const qc = useQueryClient();            // access the global cache instance

  const q = useQuery({
    queryKey: ['leads'],                  // cache bucket identifier
    queryFn: () => api.fetchLeads(page, size),
    staleTime: 60_000,                    // (optional) data is “fresh” for 60s
    refetchOnWindowFocus: false,          // (optional) prevent refetch on focus
  });
```

- `useQuery` returns `{ data, isLoading, isError, error, refetch, ... }`.
- Each render uses cached data first, then React Query decides if/when to refetch.

Mutations

```
const create = useMutation({
  mutationFn: api.createLead,
  onSuccess: (created) => {
    qc.setQueryData(['leads'], (old = []) => [...old, created]); // optimistic
cache append
    toast.success('Lead created');
  },
  onSettled: () => qc.invalidateQueries({ queryKey: ['leads'] }),
});
```

- `mutationFn` performs the API call.
- `onSuccess` updates cache immediately for snappy UI.
- `onSettled` invalidates so the next render refetches the authoritative list.

Infinite scroll mutation

```
const fetchPage = useMutation({
  mutationFn: ({ page, size }) => api.fetchLeads(page, size),
  onSuccess: (data) => {
    qc.setQueryData(['leads'], (old = []) => {
      const combined = [...old, ...(data ?? [])];
      const deduped = Array.from(
        new Map(combined.filter(Boolean).map(item => [item.id, item])).values()
      );
      return deduped;
```

```
    });
    toast.success('Page fetched');
  },
});
```

Line-by-line:

1. Combine old cache with new page.
2. Wrap in `Map` keyed by `id` → automatically drops duplicates.
3. Convert back to array → new cache snapshot.

---

5. Rendering behavior with React Query

When does React Query re-render components?

1. **Initial mount** – component subscribes; if cache has data, synchronous render happens with stale data, followed by background refetch (if stale).
2. **Data change** – any mutation or refetch that changes the cached data triggers re-render of all components subscribing to `['leads']`.
3. **Status change** – `isLoading`, `isError`, etc., change as the query transitions between states.

Because React Query stores data outside of React component state, unmounting a component does **not** drop the cache. When you navigate away and back, `useQuery` finds cached data and avoids immediate network calls, so UI feels instant.

---

6. What is Zustand?

Zustand is a tiny state management library for UI/local state:

- No reducers or action types—just plain functions.
- Uses subscribe/notify under the hood.
- Components select exactly the slices they care about, minimizing re-renders.

Store definition

```
import { create } from 'zustand';

export const useUIStore = create((set) => ({
  selectedLeadId: undefined,
  setSelected: (id) => set({ selectedLeadId: id }),

  isLeadModalOpen: false,
```

```
  setLeadModalOpen: (v) => set({ isLeadModalOpen: v }),
}));
```

Explanation:

1. `create` returns a hook (`useUIStore`) that components call.
2. The store object defines state fields and setter helpers.
3. The setter merges partial state, similar to `setState` in class components.

Rendering effect

- Components use selectors (`useUIStore(s => s.isLeadModalOpen)`).
- Only the components that read a field re-render when that field changes—other parts of the tree stay untouched.

---

7. Installing & wiring Zustand

```
cd client
npm install zustand
```

Usage in a component:

```
import { useUIStore } from '../store/uiStore';

const isOpen = useUIStore(state => state.isLeadModalOpen);
const toggle = useUIStore(state => state.setLeadModalOpen);
```

Because the store lives outside React, reloading or navigating away does not reset it unless you intentionally clear it.

---

8. End-to-end flow: `LeadsPage` walkthrough

File: `client/src/pages/LeadsPage.jsx`

```
const { q, create, remove, update, fetchPage } = useLeads();
const isOpen = useUIStore(s => s.isLeadModalOpen);
const setLeadModalOpen = useUIStore(s => s.setLeadModalOpen);
```

1. `useLeads()` subscribes to the `['leads']` query.
2. `q` exposes `data/isLoading/isError`.

3. `create/remove/update/fetchPage` mutate server state.
4. `useUIStore` controls modal visibility.

Form handling

```
const loadForm = () => {
  if (initialData) {
    return <LeadForm onSubmit={(data) => update.mutateAsync({ id:
initialData.id, data })} initialData={initialData} />;
  }
  return <LeadForm onSubmit={(data) => create.mutateAsync(data)} />;
};
```

- If `initialData` exists, the form performs an update mutation.
- Otherwise it calls the create mutation.
- Both mutation promises return to React Query, which updates the cache and re-renders the list.

Infinite scroll hook-up

```
useEffect(() => {
  if (!loadMoreRef.current) return;
  const observer = new IntersectionObserver(async ([entry]) => {
    if (!entry.isIntersecting || !hasMore || fetchPage.isPending) return;
    const newData = await fetchPage.mutateAsync({ page: page + 1, size:
pageSize });
    setPage((p) => p + 1);
    if (!newData || newData.length < pageSize) setHasMore(false);
  }, { root: parentRef.current ?? null, rootMargin: '20px 0px', threshold: 0 });

  observer.observe(loadMoreRef.current);
  return () => observer.disconnect();
}, [hasMore, fetchPage.isPending, page, q.isSuccess]);
```